

Maverick Feedback 360 — Detailed Project Analysis

This document provides an in-depth summary of the Designathon project, including technical architecture, implementation details, feature coverage, database design, frontend/back-end behavior, and demo-ready talking points. It is based on the workspace files and route implementations found in the repository.

Project Overview

Maverick Feedback 360 is a full-stack application for training feedback management. It supports separate Admin, Maverick, and Supervisor roles, and implements training session management, feedback cycles, analytics, notifications, and audit logging.

Tech Stack

Backend

- Node.js with Express for REST API server
- Prisma ORM with SQLite database for data storage
- JWT-based authentication with jsonwebtoken
- Password hashing using bcryptjs
- CORS and JSON request handling through express middleware
- File upload via multer and Excel support via xlsx
- Email/notification stubs using custom service and nodemailer package
- Development support with nodemon and dotenv configuration

Frontend

- React 19 for the user interface
- Vite as the build and dev server
- React Router DOM for client-side routing
- Axios for API communication
- Tailwind CSS for utility-based styling
- Lucide-react icons for UI elements
- React Hot Toast for toast notifications
- Recharts for analytics/charting use cases

Architecture and Structure

Backend

The backend is organized into modular route files. The main server entry point is `backend/src/server.js`, which configures CORS, JSON parsing, and registers route modules for auth, users, master data, participants, feedback, analytics, notifications, and audit logs.

Core services are implemented in `backend/src/services` and `backend/src/utls`, while the Prisma schema and seed data are stored in `backend/prisma`. Authentication middleware protects routes and enforces role-based access.

Frontend

The frontend uses React and a context-based authentication approach. `App.jsx` configures

BrowserRouter, protected routes, and role-specific route access. Layout.jsx renders a sidebar navigation based on the signed-in user role and wraps pages in a consistent layout.

AuthContext.jsx stores user state and token information in localStorage. Axios interceptors automatically attach the token to API requests and redirect the user to login on unauthorized responses.

Key Backend Features

Authentication

- Standard email/password login route
- OTP request and verification flow for alternate login
- User details endpoint returning id, name, email, role, employeeId, supervisorId
- JWT tokens with 8-hour expiry support

Role-based Access Control

Admin-only routes are enforced via authorization middleware. Routes also verify ownership on user-specific resources, so regular users cannot access other users' private feedback and forms.

User Management

- Admin can list users with paging, filtering by role or status
- Create users with hashed passwords
- Update user profiles and supervisor mappings
- Activate or deactivate users with status changes
- Audit logging on admin actions

Training Management

- CRUD operations for courses, trainers, and sessions
- Session conflict detection for trainer schedule overlap
- Participant upload by Excel template for bulk enrollment
- Participant supervisor mapping and notification on enrollment

Feedback Workflows

- Maverick feedback forms support draft saving and final submission
- Supervisor evaluation forms support draft saving and final submission
- Feedback cycles automatically close when completion threshold is met
- Admin override closing of cycles with reason and audit log

Analytics and Reporting

- Dashboard overview metrics for active sessions, overdue cycles, closed cycles, and total submissions
- Session scorecards including average rating, completion percentage, sentiment distribution, and theme clusters
- Trainer performance endpoints returning rating trends
- Leaderboards for Mavericks and Trainers
- Report generation with JSON or Excel export

Notifications and Audit

- Notification list and pagination per user

- Mark individual notifications as read or mark all as read
- Audit logs for admin activity with actor, action, entity type, and payload
- Audit routes available to Admin role only

Data Model Highlights

The Prisma schema defines the following key entities: User, Course, Trainer, Session, SessionParticipant, FeedbackCycle, MaverickFeedbackForm, SupervisorEvaluationForm, SentimentResult, ThemeCluster, Notification, AuditLog, TrainerRecommendation, and OtpCode.

This model supports rich relationships between sessions, enrollments, feedback cycles, and roles, enabling the full feedback lifecycle.

Frontend Pages and Role Experience

Admin

- Dashboard with metrics, recent sessions, and alerts
- Courses management page
- Trainers management page
- Sessions management and detail views
- Analytics and performance insights
- Reports view for export
- Leaderboard and user audit log

Maverick

- Personal dashboard
- Feedback form list and submission flow
- Training history page
- Leaderboard access

Supervisor

- Supervisor dashboard
- Evaluations list and detailed evaluation forms
- Managed mavericks list
- Leaderboard access

Demo Talking Points

- Show login and authentication flow for a role-based user
- Demonstrate Admin dashboard metrics and recent alerts
- Create or edit a course, trainer, and session
- Upload participants with Excel template and enroll users
- Complete a Maverick feedback submission and show its status
- Complete a Supervisor evaluation and trigger analytics
- Display analytics charts, leaderboards, and session scorecards
- Open the audit log to show traceability and compliance

Commands

- Backend: `cd backend && npm install && npm run dev`
- Frontend: `cd frontend && npm install && npm run dev`
- Seed database: `cd backend && npm run seed`

The application is ready for a product demo that highlights training feedback lifecycle, role-based workflows, analytics, and admin governance. The backend is modular and extensible, while the frontend supports quick navigation and real-time feedback for each user role.